

DevOps + MLOps Practice Lab

Learner Manual

16 hands-on labs: from a Python script to a monitored, auto-scaling, self-retraining ML system on Kubernetes.

Repository: github.com/KKRavindu/devops-mlops-lab

Version 1.0 · October 2026

Contents

Part I: Introduction . . . 3

1. Welcome . . . 3
2. What you will learn . . . 3
3. How the whole system fits together . . . 4
4. The application you will operate . . . 4
5. Repository map . . . 5
6. Before you start . . . 5
7. How every lab is organised . . . 6
8. Learning path and time plan . . . 6

Part II: The Labs . . . 7

Lab 00: Setup & Git Workflow . . . 8
Lab 01: Run, Test and Understand the App . . . 9
Lab 02: Docker: Images, Layers, Multi-stage, Security . . . 10
Lab 03: Docker Compose: A Multi-service Stack . . . 11
Lab 04: CI/CD with GitHub Actions . . . 12
Lab 05: Kubernetes Fundamentals . . . 13
Lab 06: Kustomize Environments, Config and Scaling . . . 14
Lab 07: Infrastructure as Code with Terraform . . . 15
Lab 08: Experiment Tracking & Model Registry (MLflow) . . . 16
Lab 09: Reproducible Pipelines & Data Versioning (DVC) . . . 17
Lab 10: Continuous Training (CT) . . . 18
Lab 11: Observability: Metrics, Dashboards, Alerts . . . 19
Lab 12: Data Drift Detection & the Retraining Loop . . . 20
Lab 13: Release Strategies: Rolling, Canary, Rollback . . . 21
Lab 14: GitOps with Argo CD . . . 22
Lab 15: Capstone: Incident Drills & Chaos Engineering . . . 23

Part III: Reference . . . 24

A. Command cheat sheet . . . 24
B. Troubleshooting . . . 25
C. Glossary . . . 26
D. Progress tracker . . . 27

Part I: Introduction

1. Welcome

This manual guides you through the **DevOps + MLOps Practice Lab**, a hands-on platform for learning to build, ship and operate machine-learning systems the way professional engineering teams do.

You work with **one real application** the whole way through: a small machine-learning service that predicts the grape variety of a wine from 13 chemical measurements. The model is simple on purpose. Everything around the model is what you are here to learn: testing, containers, pipelines, Kubernetes, infrastructure as code, monitoring, drift detection, retraining, safe releases and incident response.

By the end you will have a portfolio project that demonstrates the full lifecycle of a production ML system, plus written notes and postmortems you can use for interviews.

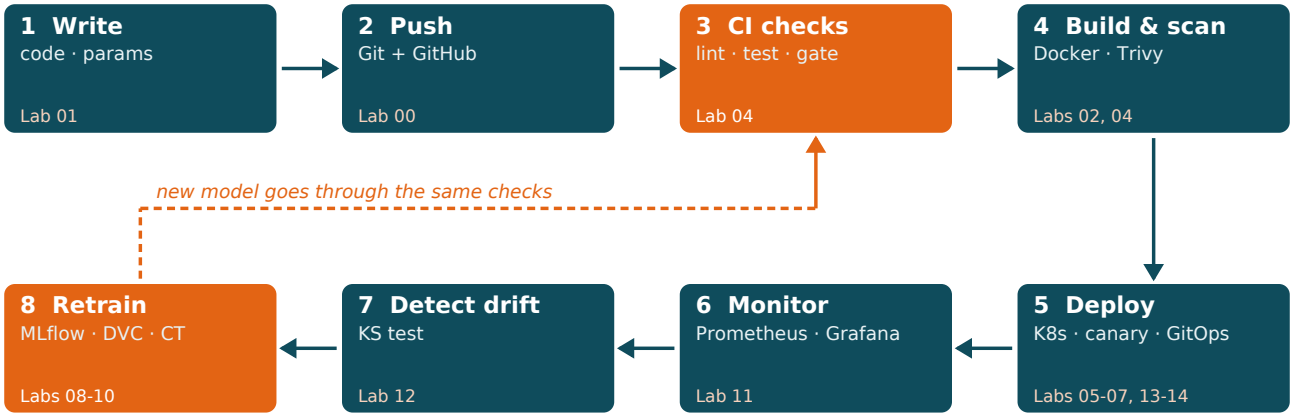
> **Who this manual is for**
>
> - Developers, sysadmins and data scientists moving into **DevOps** or **MLOps** roles.
> - Students who know basic Python and the Linux command line and want real practice, not just theory.
> - Anyone preparing for interviews or certifications such as CKA/CKAD or Terraform Associate.

2. What you will learn

Skill area	Tools you will use	Labs
Version control & collaboration	Git, GitHub, pull requests	00
Testing & code quality	pytest, ruff, Makefile	01
Containers	Docker, multi-stage builds, Trivy	02
Multi-service systems	Docker Compose	03
Continuous integration & delivery	GitHub Actions, GHCR	04
Container orchestration	Kubernetes (minikube), kubectl	05
Environments & autoscaling	Kustomize, HPA, PDB	06
Infrastructure as Code	Terraform	07
Experiment tracking & model registry	MLflow	08
Data & pipeline versioning	DVC	09
Continuous training	GitHub Actions (scheduled)	10
Observability	Prometheus, Grafana, PromQL, alerts	11
Model monitoring	Statistical drift detection (KS test)	12
Release engineering	Rolling, canary, rollback	13
GitOps	Argo CD	14
Operations & incident response	Chaos drills, postmortems	15

3. How the whole system fits together

Every lab builds one piece of the loop below. A change starts on your laptop, is checked automatically, packaged, deployed and monitored. When real-world data drifts away from the training data, the model is retrained, and the new model goes through exactly the same checks as any code change.



Ordinary software has one moving part: **code**. Machine-learning systems have three: **code, data and the trained model**. A model can become worse even when nobody touches the code, simply because the world changes. That is why this platform has two pipelines: **CI/CD** for the software and **Continuous Training (CT)** for the model.

4. The application you will operate

The ML pipeline has two stages. `src/prepare.py` splits the dataset into training and test sets. `src/train.py` trains the model, measures its accuracy, and refuses to save it if accuracy is below the **quality gate** (85% by default, set in `params.yaml`).

The trained model is served by a FastAPI web service, `src/serve.py`, with these endpoints:

Endpoint	Purpose
`GET /health`	Liveness: the process is running.
`GET /ready`	Readiness: the model is loaded and the service can take traffic.
`GET /model`	Metadata: version, model type, accuracy, training time.
`POST /predict`	Send 13 numbers per row, receive a class and a confidence score.
`GET /metrics`	Prometheus metrics: requests, latency, predictions, confidence.
`GET /docs`	Interactive API documentation (Swagger UI).

Two built-in **fault switches** let you practise incident response safely: `FAULT_LATENCY_MS` adds artificial delay and `FAULT_ERROR_RATE` makes a fraction of requests fail. Every prediction can also be written to a log file, which the drift detector later compares against the training data.

```
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' \
  -d '{"features":[[13.2,1.78,2.14,11.2,100,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050]]}'

{"model_version":"...","predictions":[{"class":"class_0","confidence":0.97}]}
```

5. Repository map

Path	What it contains
`src/`	The ML pipeline (prepare, train) and the API (serve).
`tests/`	11 automated tests for the pipeline and the API.
`params.yaml`	Every tunable setting of the pipeline, including the quality gate.
`dvc.yaml`	The reproducible pipeline definition used by DVC.
`Dockerfile`	Multi-stage image: install, train, then a slim non-root runtime.
`docker-compose.yml`	API + MLflow + Prometheus + Grafana on one network.
`monitoring/`	Prometheus scrape config, alert rules, Grafana dashboard.
`k8s/`	Kubernetes base, dev and prod overlays, canary, Argo CD app.
`terraform/`	Namespaces, quotas and default limits as code.
`.github/workflows/`	`ci.yml` (CI/CD) and `ml-pipeline.yml` (continuous training).
`scripts/`	Traffic simulator (with optional drift) and the drift detector.
`labs/`	The detailed instructions for each lab (00 to 15).
`Makefile`	Short commands for everything. Run `make help` to list them.

6. Before you start

Computer

- Linux (or macOS / Windows with WSL2), **at least 8 GB RAM** (16 GB recommended), 4+ CPU cores and about 15 GB of free disk space.
- The Kubernetes labs give minikube 6 GB. Stop the Docker Compose stack (`make down`) before starting minikube so both don't compete for memory.

Tools

Tool	Needed from	How to get it
git, make, Python 3.10+	Lab 00	Usually pre-installed on Linux.
Docker + Docker Compose	Lab 02	docs.docker.com/engine/install
kubectl + minikube	Lab 05	kubernetes.io/docs/tasks/tools
Terraform	Lab 07	developer.hashicorp.com/terraform/install
GitHub CLI (`gh`)	Lab 10	cli.github.com, then run `gh auth login`
Argo CD CLI (optional)	Lab 14	argo-cd.readthedocs.io

Accounts

- A **GitHub** account. The CI/CD, continuous training and GitOps labs run on GitHub.
- Everything else runs locally and is free.

7. How every lab is organised

Each lab file in `labs/` follows the same structure, so you always know what to do next:

Section	What you do there
Goal	One sentence describing what you will be able to do afterwards.
Theory	The concepts behind the lab. Read this first; it is short.
Tasks	Step-by-step commands. Type them yourself and read the output.
Checkpoint	How to prove the lab is complete. Don't move on until it passes.
Break it	Deliberately cause a failure, then diagnose and fix it.
Challenges	Optional extensions that take you beyond the tutorial.
Interview questions	Answer them in your notes, in your own words.

How to get the most out of the labs

- **Type, don't paste**, at least the first time. Muscle memory matters.
- **Do the 'Break it' sections.** Real DevOps work is mostly debugging, and these drills are where you learn it.
- **Keep notes** in a `notes/` folder in the repo: what you did, what broke, what you learned. These become your interview preparation.
- **Commit after every lab** with a clear message. Your Git history becomes your portfolio.
- **Don't rush.** Understanding one lab well beats skimming three.

8. Learning path and time plan

The labs are grouped into four phases. Times are estimates for a first pass, including the 'Break it' drills but not the optional challenges.

Phase	Labs	Focus	Time
1. Foundations	00 to 03	Git, the app, Docker, Compose	about 8 h
2. Delivery	04 to 07	CI/CD, Kubernetes, Kustomize, Terraform	about 12 h
3. MLOps	08 to 10	MLflow, DVC, continuous training	about 7 h
4. Operate	11 to 15	Monitoring, drift, releases, GitOps, incidents	about 17 h
Total	16 labs		**about 44 h**

A comfortable pace is **two labs per week**, finishing in about two months. Labs must be done in order, because each one builds on the previous ones.

Part II: The Labs

Each lab below is summarised in the same format: **why it matters**, **what you will do**, the **key commands**, how you know you are **done**, and the common mistakes to **watch out** for. The full instructions, theory and interview questions are in the lab file named at the end of each section.

Run all commands from the project folder `~/Projects/devops-mlops-lab` unless a step says otherwise.

Lab	Title	Phase	Time
00	Setup & Git Workflow	Foundations	1 h
01	Run, Test and Understand the App	Foundations	2 h
02	Docker: Images, Layers, Multi-stage, Security	Foundations	2 to 3 h
03	Docker Compose: A Multi-service Stack	Foundations	2 h
04	CI/CD with GitHub Actions	Delivery	3 h
05	Kubernetes Fundamentals	Delivery	3 to 4 h
06	Kustomize Environments, Config and Scaling	Delivery	3 h
07	Infrastructure as Code with Terraform	Delivery	3 h
08	Experiment Tracking & Model Registry (MLflow)	MLOps	2 to 3 h
09	Reproducible Pipelines & Data Versioning (DVC)	MLOps	2 to 3 h
10	Continuous Training (CT)	MLOps	2 h
11	Observability: Metrics, Dashboards, Alerts	Operate	3 h
12	Data Drift Detection & the Retraining Loop	Operate	2 to 3 h
13	Release Strategies: Rolling, Canary, Rollback	Operate	3 h
14	GitOps with Argo CD	Operate	3 to 4 h
15	Capstone: Incident Drills & Chaos Engineering	Operate	4 to 6 h

LAB
00

Setup & Git Workflow

Phase 1 · Foundations · Estimated time: 1 h

Why this matters

Git is the single source of truth for everything in DevOps: code, infrastructure, pipelines and configuration. Every later lab relies on the branch, pull request and merge workflow you practise here.

What you will do

1. Turn the project into a Git repository and make the first commit.
2. Create an empty repository on GitHub and push to it. **(You have already done this.)**
3. Create the Python environment with `make setup`.
4. Practise the pull-request flow: create a branch, add a `notes/README.md`, push the branch, open a PR on GitHub, review and merge it, then pull `main`.
5. Optionally protect the `main` branch (read the warning below first).

Key commands

```
make setup
git switch -c docs/my-notes
mkdir notes && echo "# My lab notes" > notes/README.md
git add notes && git commit -m "docs: start lab notes"
git push -u origin docs/my-notes      # then open and merge the PR on GitHub
git switch main && git pull
```

You are done when

- ☐ Your repository is on GitHub.
- ☐ One pull request has been merged.
- ☐ `make help` lists all commands.

Watch out

- GitHub does not accept your account password for `git push`. Use `gh auth login` or a personal access token with the `repo` and `workflow` scopes.
- If you require pull requests on `main`, the CI job `gitops-bump` can no longer push its commit. Skip branch protection for now, or allow GitHub Actions to bypass it.
- After every green CI run on `main`, a bot commit is added. Run `git pull` before you push again.

Full instructions, theory, challenges and interview questions: `labs/00-setup.md`

LAB 01

Run, Test and Understand the App

Phase 1 · Foundations · Estimated time: 2 h

Why this matters

You cannot operate what you don't understand. This lab shows exactly what the service does, how the model is produced, what the quality gate protects against, and the difference between a service that is alive and one that is ready.

What you will do

1. Run the ML pipeline and inspect the data, the model metadata and `metrics.json`.
2. Run the tests and the linter.
3. Start the API and call each endpoint with `curl`. Open the Swagger UI at `http://localhost:8000/docs`.
4. Generate traffic and watch the prediction log grow.
5. Break it: raise the quality gate to 0.99, delete the model folder, and send the wrong number of features. Explain each result.

Key commands

```
make pipeline && cat models/metadata.json
make test && make lint
make serve                                  # terminal 1
curl localhost:8000/ready                   # terminal 2
curl -s localhost:8000/metrics | grep -E '^(api|model)_'
make traffic && tail -1 logs/predictions.jsonl
```

You are done when

- ☐ All 11 tests pass.
- ☐ You can explain what each test protects against.
- ☐ You can explain `/health` vs `/ready` and every metric on `/metrics`.

Watch out

- If `pytest` fails with errors from `/opt/ros/...`, your shell's `PYTHONPATH` points at ROS. Use the `make` commands (they ignore it) or run `unset PYTHONPATH`.
- `make serve` keeps the terminal busy. Use a second terminal for `curl`.

Full instructions, theory, challenges and interview questions: `labs/01-local-app.md`

LAB 02

Docker: Images, Layers, Multi-stage, Security

Phase 1 · Foundations · Estimated time: 2 to 3 h

Why this matters

Containers make software run the same everywhere. A good image is small, fast to build, reproducible and secure. These are basic expectations in any DevOps role.

What you will do

1. Build the image and run it.
2. Inspect it: layer history, the user it runs as (should be UID 10001, not root), and its health check.
3. Observe the build cache: change a code file and rebuild, then change `requirements.txt` and rebuild. Compare which steps are cached.
4. Build only the trainer stage with `--target trainer`.
5. Build the **v2 image** (logistic regression). You need it in Lab 13.
6. Scan the image for vulnerabilities with Trivy.

Key commands

```
make docker-build
docker history wine-api:latest
docker run --rm wine-api:latest id          # uid=10001(appuser)
docker build --build-arg TRAIN_MODEL=logistic_regression \
  --build-arg MODEL_VERSION=v2 -t wine-api:v2 .
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:0.56.2 image --severity HIGH,CRITICAL wine-api:latest
```

You are done when

- ☐ The image runs as a non-root user.
- ☐ `docker ps` shows the container as `(healthy)`.
- ☐ You can explain what each of the three build stages does.

Watch out

- `permission denied` on the Docker socket: add yourself to the `docker` group (`sudo usermod -aG docker $USER`), then log out and back in.
- Never put secrets in a Dockerfile `ENV`. The 'Break it' task shows how easily they can be read back.

Full instructions, theory, challenges and interview questions: `labs/02-docker.md`

LAB 03

Docker Compose: A Multi-service Stack

Phase 1 · Foundations · Estimated time: 2 h

Why this matters

Real systems are several services working together. Compose runs the whole platform on your laptop and teaches service discovery, volumes and configuration through environment variables.

What you will do

1. Start the stack and open the four web interfaces (table below).
2. Send traffic and watch the Grafana dashboard come alive.
3. Explore networking: containers reach each other by service name, not `localhost`.
4. Change configuration without rebuilding: add 300 ms latency with `FAULT_LATENCY_MS`, then remove it.
5. Break it: stop the API and watch the `ApiDown` alert fire in Prometheus.

Service	URL	Purpose
API	`http://localhost:8000/docs`	The model service
MLflow	`http://localhost:5000`	Experiments and model registry
Prometheus	`http://localhost:9090`	Metrics, targets and alerts
Grafana	`http://localhost:3000`	Dashboards (MLOps Lab folder)

Key commands

```
make up
make traffic
docker compose exec prometheus wget -qO- http://api:8000/health
FAULT_LATENCY_MS=300 docker compose up -d api     # slow it down
docker compose up -d api                          # back to normal
make logs
```

You are done when

- ☐ All four services are running.
- ☐ The Prometheus target is **UP**.
- ☐ The Grafana dashboard shows live data.

Watch out

- 'Port already in use': another program is using port 8000, 5000, 9090 or 3000. Stop it or change the port in `docker-compose.yml`.
- `docker compose down -v` also deletes the MLflow data volume.
- Run `make down` when you're finished, before starting minikube.

Full instructions, theory, challenges and interview questions: `labs/03-compose.md`

LAB
04

CI/CD with GitHub Actions

Phase 2 · Delivery · Estimated time: 3 h

Why this matters

CI/CD automatically checks every change and turns it into a release you can trace. It replaces error-prone manual steps with the same checks, every time.

What you will do

1. Read `.github/workflows/ci.yml` and identify its three jobs: `lint-test`, `build-scan-push` and `gitops-bump`.
2. Open the **Actions** tab in your repository and follow a run.
3. Find your image under **Packages** and pull it by its `sha-...` tag.
4. Open a pull request that breaks a test, confirm CI turns red, and make the check required so a red PR cannot be merged.
5. Run the same checks locally before pushing.

CI job	What it does
`lint-test`	Installs dependencies, runs `ruff` and the 11 tests.
`build-scan-push`	Builds the image, scans it with Trivy, smoke-tests the container, and pushes it to GHCR (on `main` only).
`gitops-bump`	Writes the new image tag into `k8s/overlays/prod` and commits it. This is the hand-off to GitOps in Lab 14.

Key commands

```
make lint && make test && make docker-build    # local version of CI
gh run list                                    # recent pipeline runs
gh run watch                                   # follow the current run
docker pull ghcr.io/kkravindu/wine-api:sha-<commit>
```

You are done when

- ☐ A green pipeline on `main`.
- ☐ An image in GHCR tagged with the commit SHA.
- ☐ A red pull request that could not be merged.

Watch out

- Workflow permissions must be **Read and write** (repo Settings, then Actions, then General). Otherwise pushing the image fails.
- Trivy can fail when a new vulnerability is published for the base image. That means the scan is working. Read the report and update the affected package or base image.

Full instructions, theory, challenges and interview questions: `labs/04-ci-cd.md`

LAB
05

Kubernetes Fundamentals

Phase 2 · Delivery · Estimated time: 3 to 4 h

Why this matters

Kubernetes is the standard platform for running containers in production. You will learn how it keeps your service running, replaces failed pods, and decides when a pod may receive traffic.

What you will do

1. Start minikube and load your image into it.
2. Deploy the dev environment and look at the Deployment, ReplicaSet, Pods and Service it creates.
3. Reach the API through a port-forward.
4. Explore with `describe`, `logs`, `exec` and `get events`.
5. Delete a pod and watch Kubernetes replace it (self-healing).
6. Break it: cause ImagePullBackOff, OOMKilled, a pod that never becomes ready, and CrashLoopBackOff. Diagnose each one.

Key commands

```
make down                    # free memory first
make k8s-start && make k8s-load
make k8s-deploy ENV=dev && make k8s-status ENV=dev
make k8s-forward ENV=dev     # terminal 1
curl localhost:8080/ready    # terminal 2
kubectl -n wine-dev describe pod -l app=wine-api
kubectl -n wine-dev get events --sort-by=.lastTimestamp
kubectl -n wine-dev rollout undo deploy/wine-api     # fix a broken change
```

You are done when

- ☐ `/ready` answers through the port-forward.
- ☐ You can draw the chain Deployment, ReplicaSet, Pod, Service.
- ☐ You diagnosed all four 'Break it' failures from events and logs.

Watch out

- **Stale images:** the manifests use `wine-api:latest` with `imagePullPolicy: IfNotPresent`. After rebuilding, run `make k8s-load` again and `kubectl -n wine-dev rollout restart deploy/wine-api`, or Kubernetes keeps the old image.
- A port-forward connects to a single pod. It is fine for testing, but it doesn't show load balancing.

Full instructions, theory, challenges and interview questions: `labs/05-kubernetes.md`

LAB
06

Kustomize Environments, Config and Scaling

Phase 2 · Delivery · Estimated time: 3 h

Why this matters

Real teams run the same application in several environments with different settings. Kustomize does this without copying YAML. Autoscaling handles traffic spikes without anyone getting paged.

What you will do

1. Render the dev and prod overlays and compare them with `diff`.
2. Change a setting in the dev ConfigMap and watch Kubernetes roll out new pods automatically (thanks to the hash suffix).
3. Generate load and watch the Horizontal Pod Autoscaler add replicas.
4. Deploy the prod overlay and compare the replica counts.
5. Break it: remove resource requests and see the HPA show `<unknown>`. Simulate a node drain with the PodDisruptionBudget in place.

Key commands

```
kubectl kustomize k8s/overlays/dev  > /tmp/dev.yaml
kubectl kustomize k8s/overlays/prod > /tmp/prod.yaml
diff /tmp/dev.yaml /tmp/prod.yaml
kubectl -n wine-dev get hpa -w                # watch scaling
make k8s-deploy ENV=prod
```

You are done when

- ☐ You can explain every difference between dev and prod.
- ☐ You saw the HPA add replicas under load.

Watch out

- The HPA shows `<unknown>` for about a minute after metrics-server starts. Wait before you conclude something is broken.
- The HPA calculates CPU usage relative to resource **requests**. Without requests, it cannot scale.

Full instructions, theory, challenges and interview questions: `labs/06-kustomize-scaling.md`

LAB
07

Infrastructure as Code with Terraform

Phase 2 · Delivery · Estimated time: 3 h

Why this matters

Infrastructure defined in code can be reviewed, repeated and audited. Terraform is the most widely used IaC tool, and its plan/apply/state model shows up in almost every DevOps interview.

What you will do

1. Install Terraform.
2. Hand the namespaces over to Terraform: delete the ones `make k8s-deploy` created, then run `init`, `validate`, `plan` and `apply`.
3. Inspect the ResourceQuota and LimitRange that Terraform created.
4. Add a `staging` environment to `variables.tf` and see that `plan` only adds the new resources.
5. Change a namespace label by hand and watch `terraform plan` detect the drift and `apply` revert it.

Key commands

```
make k8s-delete ENV=dev; make k8s-delete ENV=prod
kubectl delete ns wine-dev wine-prod --ignore-not-found --wait
cd terraform
terraform init && terraform validate
terraform plan && terraform apply
kubectl -n wine-dev describe quota,limitrange
kubectl label ns wine-dev managed-by=me --overwrite && terraform plan
```

You are done when

- ☐ `terraform plan` reports 'No changes' after apply.
- ☐ You can explain what the state file stores and why it is sensitive.

Watch out

- 'object is being deleted: namespace already exists' means the old namespace is still terminating. Wait with `kubectl wait --for=delete ns/wine-dev`, then apply again.
- Never let two tools manage the same object. Terraform owns the namespaces, and Kustomize or Argo CD owns what runs inside them.
- The state file here is local. Real teams store it remotely with locking.

Full instructions, theory, challenges and interview questions: `labs/07-terraform.md`

LAB
08

Experiment Tracking & Model Registry (MLflow)

Phase 3 · MLOps · Estimated time: 2 to 3 h

Why this matters

'Which model is in production, and how exactly was it trained?' Without experiment tracking nobody can answer that. MLflow records every run and gives models versions and aliases such as `champion`.

What you will do

1. Start the Compose stack so MLflow is running.
2. Point training at MLflow and train several variants with different model types and parameters.
3. Compare the runs side by side in the MLflow UI.
4. In the model registry, set the `champion` alias on the best version.
5. Load the champion model from Python by its alias.

Key commands

```
make up
export MLFLOW_TRACKING_URI=http://localhost:5000
make train
TRAIN_MODEL=logistic_regression make train
# edit params.yaml (e.g. n_estimators: 10, max_depth: 2) and train again
.venv/bin/python -c "import mlflow; mlflow.set_tracking_uri('http://localhost:5000'); \
print(mlflow.sklearn.load_model('models:/wine-classifier@champion'))"
```

You are done when

- ☐ At least 4 runs are recorded.
- ☐ The registry holds 3 or more versions.
- ☐ A `champion` alias exists and loads from Python.

Watch out

- `export` only applies to the current terminal. In a new terminal, set `MLFLOW_TRACKING_URI` again.
- Set `params.yaml` back to its original values afterwards, or later labs will use a weaker model.

Full instructions, theory, challenges and interview questions: `labs/08-mlflow.md`

LAB
09

Reproducible Pipelines & Data Versioning (DVC)

Phase 3 · MLOps · Estimated time: 2 to 3 h

Why this matters

Git can't store large datasets and models, but you still need to know exactly which data produced which model. DVC versions them through small pointer files and re-runs only the pipeline stages whose inputs changed.

What you will do

1. Initialise DVC and configure a local remote.
2. Look at the pipeline graph and run it.
3. Commit `dvc.lock` and `metrics.json`, then push the data with `dvc push`.
4. Run `dvc repro` again (nothing runs). Change `max_depth` and run it again (only `train` runs).
5. Compare parameters and metrics across commits, then restore an old model ('time travel').

Key commands

```
dvc init && dvc remote add -d localremote /tmp/dvc-store
dvc dag && dvc repro
git add .dvc dvc.yaml dvc.lock metrics.json && git commit -m "ml: baseline model"
dvc push
# change train.max_depth in params.yaml
dvc repro && dvc params diff && dvc metrics diff
```

You are done when

- ☐ `dvc repro` does nothing on an unchanged project.
- ☐ You can restore a past model from Git history.

Watch out

- Activate the virtual environment first (`source .venv/bin/activate`), since DVC is installed there.
- Commit `metrics.json` to Git, or `dvc metrics diff` has nothing to compare against.

Full instructions, theory, challenges and interview questions: `labs/09-dvc.md`

LAB 10

Continuous Training (CT)

Phase 3 · MLOps · Estimated time: 2 h

Why this matters

Models go stale. Continuous training retrains them automatically on a schedule, when settings change, or on demand, and only publishes models that pass the quality gate.

What you will do

1. Read `.github/workflows/ml-pipeline.yml` and list its four triggers.
2. Run the pipeline manually with the `logistic_regression` model.
3. Read the metrics in the run summary and download the model artifact.
4. Change `max_depth` in `params.yaml`, push, and confirm retraining starts by itself.
5. Push a change that fails the quality gate and confirm no model is published. Then revert it.

Key commands

```
gh workflow run "ML Pipeline" -f model=logistic_regression
gh run watch
gh run download <run-id>            # fetch the model artifact
```

You are done when

- ☐ Three successful runs, each from a different trigger.
- ☐ One run correctly blocked by the quality gate.

Watch out

- Revert the failing quality-gate change straight away. It also makes CI fail.
- The scheduled run happens every Monday at 03:00 UTC. You can't make it run sooner; use the manual trigger to test.

Full instructions, theory, challenges and interview questions: `labs/10-continuous-training.md`

LAB 11

Observability: Metrics, Dashboards, Alerts

Phase 4 · Operate · Estimated time: 3 h

Why this matters

You can't fix what you can't see. ML adds a twist: a model can return HTTP 200 with wrong answers. So you monitor the model's behaviour as well as the service's health.

What you will do

1. Run the stack with traffic and write PromQL queries for request rate, p95 latency and predictions per class.
2. Read every alert rule in `monitoring/alert_rules.yml`.
3. Fire **HighErrorRate** by setting `FAULT_ERROR_RATE=0.2` and sending traffic.
4. Fire **HighLatencyP95** by setting `FAULT_LATENCY_MS=700`.
5. Fire an **ML alert** by sending heavily drifted traffic. The service still looks healthy while confidence drops.

Key commands

```
make up && make traffic
FAULT_ERROR_RATE=0.2 docker compose up -d api
.venv/bin/python scripts/simulate_traffic.py --n 2000 --rps 20
docker compose up -d api                          # reset
.venv/bin/python scripts/simulate_traffic.py --n 2000 --drift 1.5
```

You are done when

- ☐ You triggered one service alert and one model alert.
- ☐ You can explain every PromQL expression you used.

Watch out

- Alerts have a `for:` duration, so traffic must keep flowing for 1 to 2 minutes before an alert moves from *pending* to *firing*.
- Always reset the fault switches afterwards (`docker compose up -d api`).

Full instructions, theory, challenges and interview questions: `labs/11-observability.md`

LAB 12

Data Drift Detection & the Retraining Loop

Phase 4 · Operate · Estimated time: 2 to 3 h

Why this matters

Production data changes over time: new suppliers, recalibrated sensors, new customer behaviour. Drift detection notices when inputs no longer look like the training data, which is the signal to retrain.

What you will do

1. Send normal traffic and run the drift check. It should exit with code 0.
2. Send drifted traffic and run it again. It should exit with code 1 and flag exactly the four drifted features.
3. Read `drift_report.json` and explain the KS statistic and p-value in plain words.
4. Find the sensitivity limit: try smaller drift values and different sample sizes.
5. Close the loop: trigger retraining automatically when drift is detected.

Key commands

```
rm -f logs/predictions.jsonl && make traffic && make drift-check
rm -f logs/predictions.jsonl && make traffic-drift && make drift-check
echo "exit code: $?"
make drift-check || gh workflow run "ML Pipeline"
```

You are done when

- ☐ A clean run exits 0 and a drifted run exits 1.
- ☐ You can explain data drift vs concept drift.

Watch out

- Delete `logs/predictions.jsonl` between experiments, or old and new traffic get mixed together.
- The check needs at least 50 predictions before it gives a verdict.

Full instructions, theory, challenges and interview questions: `labs/12-drift.md`

LAB 13

Release Strategies: Rolling, Canary, Rollback

Phase 4 · Operate · Estimated time: 3 h

Why this matters

Every release is a risk. Professional teams limit how many users a bad release can reach (canary) and can undo it in seconds (rollback), without dropping a single request.

What you will do

1. Build the v2 model image and load it into minikube.
2. Run 3 stable pods plus 1 canary pod. About 25% of traffic reaches v2.
3. Measure the split from **inside** the cluster.
4. Promote v2 to all pods, remove the canary, then roll back.
5. Prove zero downtime: send traffic during the rollouts and count the failures (target: 0).
6. Break it: remove the `preStop` hook and watch requests fail during rollouts.

Key commands

```
minikube image load wine-api:v2
kubectl -n wine-dev scale deploy/wine-api --replicas=3
kubectl -n wine-dev patch hpa wine-api -p '{"spec":{"minReplicas":3}}'
kubectl -n wine-dev apply -f k8s/canary/deployment-canary.yaml
kubectl -n wine-dev set image deploy/wine-api api=wine-api:v2     # promote
kubectl -n wine-dev delete deploy wine-api-canary
kubectl -n wine-dev rollout undo deploy/wine-api                  # rollback
```

You are done when

- ☐ You measured a split of roughly 75/25.
- ☐ Promote and rollback completed with **0** failed requests.
- ☐ You can explain why the `preStop` hook is needed.

Watch out

- A port-forward always hits one pod, so it can't show the traffic split. Use a curl pod inside the cluster, as the lab file shows.
- If you did Lab 07, the namespace has a quota. Rollouts need room for one extra pod (`maxSurge: 1`), so the quota must include that headroom.

Full instructions, theory, challenges and interview questions: `labs/13-release-strategies.md`

LAB
14

GitOps with Argo CD

Phase 4 · Operate · Estimated time: 3 to 4 h

Why this matters

With GitOps, Git is the only way to change production. An agent inside the cluster pulls the desired state from Git and continuously corrects any manual changes. Rolling back is simply `git revert`.

What you will do

1. Install Argo CD in minikube and log in to its web UI.
2. Set `repoURL` in `k8s/argocd/application.yaml` to `https://github.com/KKRavindu/devops-mlops-lab.git`, commit, push and apply it.
3. Make the `wine-api` package public on GitHub so the cluster can pull it.
4. Merge a code change and watch it reach the cluster with no `kubectl` from you.
5. Scale the deployment by hand and watch Argo CD undo it (self-heal).
6. Roll back by reverting the bot's deploy commit.

Key commands

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/v2.12.4/manifests/install.yaml
kubectl -n argocd port-forward svc/argocd-server 8443:443
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
kubectl apply -f k8s/argocd/application.yaml
```

You are done when

- ☐ A merge to `main` reaches the cluster automatically.
- ☐ A manual change gets reverted by Argo CD.

Watch out

- ImagePullBackOff in `wine-prod` usually means the GHCR package is still private.
- Prod runs at least 3 replicas. Give minikube enough memory or reduce them for the exercise.
- With GitOps, `kubectl rollout undo` is reverted by Argo CD. Roll back through Git instead.

Full instructions, theory, challenges and interview questions: `labs/14-gitops.md`

LAB
15

Capstone: Incident Drills & Chaos Engineering

Phase 4 · Operate · Estimated time: 4 to 6 h

Why this matters

On-call engineers are judged by how calmly and quickly they restore service. These drills train the full response loop under realistic conditions, and the final project ties every lab into one demonstration.

What you will do

1. Learn the response loop: **detect, triage, mitigate, diagnose, postmortem**.
2. Run the 8 drills (ideally with a friend injecting them without telling you which). They range from error spikes and OOM kills to silent model drift and a failed node.
3. For each one, find the problem from dashboards and alerts first, restore service, then find the root cause.
4. Write a blameless postmortem for each drill in `notes/`.
5. Build the final demo: param change, CI, CT, GitOps deploy, drift alert, retraining, canary and promotion. Record a 5-minute walkthrough.

Key commands

```
shuf -i 1-8 -n 1                    # pick a random drill
kubectl -n wine-dev set env deploy/wine-api FAULT_ERROR_RATE=0.3   # drill 1
kubectl -n wine-dev set env deploy/wine-api FAULT_ERROR_RATE-      # remove it
```

You are done when

- ☐ All 8 drills resolved, each with a written postmortem.
- ☐ A recorded end-to-end demo linked from your README.

Watch out

- Drill 8 stops the kubelet. Always start it again with `minikube ssh -- sudo systemctl start kubelet`.
- Mitigate first, investigate second. Rolling back is a valid fix while you look for the root cause.

Full instructions, theory, challenges and interview questions: `labs/15-capstone-incidents.md`

Part III: Reference

A. Command cheat sheet

Command	What it does
`make help`	List every command.
`make setup`	Create `.venv` and install all dependencies.
`make pipeline`	Prepare data and train the model (with the quality gate).
`make test` / `make lint`	Run the tests / check code style.
`make serve`	Run the API locally with auto-reload on port 8000.
`make docker-build` / `make docker-run`	Build / run the container image.
`make up` / `make down`	Start / stop the full Compose stack.
`make logs`	Follow the API container logs.
`make traffic` / `make traffic-drift`	Send normal / drifted traffic.
`make drift-check`	Compare live predictions with the training data.
`make k8s-start`	Start minikube with metrics-server.
`make k8s-load`	Build the image and load it into minikube.
`make k8s-deploy ENV=dev`	Deploy an overlay (`dev` or `prod`).
`make k8s-status ENV=dev`	Show deployments, pods, services, HPA and PDB.
`make k8s-forward ENV=dev`	Port-forward the service to `localhost:8080`.
`make k8s-delete ENV=dev`	Remove an overlay from the cluster.
`make clean`	Delete generated data, models, logs and reports.

B. Troubleshooting

Symptom	Cause and fix
`pytest` crashes with errors from `/opt/ros/...`	ROS adds its packages to `PYTHONPATH`. Use `make test`, or run `unset PYTHONPATH` first.
`git push`: 'Password authentication is not supported'	Use `gh auth login` and `gh auth setup-git`, or a personal access token with the `repo` and `workflow` scopes.
`git push` rejected: 'fetch first'	The CI bot pushed a commit. Run `git pull --rebase`, then push again.
CI fails at 'Push to GHCR' or 'gitops-bump'	Set Workflow permissions to Read and write, then re-run the jobs.
`permission denied` on `/var/run/docker.sock`	`sudo usermod -aG docker $USER`, then log out and back in.
'Port is already allocated' when running `make up`	Another process uses the port. Find it with `ss -ltnp | grep 8000`.
minikube won't start or is very slow	Not enough memory. Run `make down` first, or use `minikube start --memory 4g`.
Pod still runs the old code after rebuilding	Stale `latest` image. Run `make k8s-load` again, then `kubectl -n wine-dev rollout restart deploy/wine-api`.
HPA target shows `<unknown>`	metrics-server is still starting (wait a minute), or the pod has no resource requests.
Pods 'forbidden: exceeded quota'	The Terraform ResourceQuota is full. Scale down or raise `cpu_limit` in `terraform/variables.tf`.
Terraform: 'object is being deleted'	The namespace is still terminating. Run `kubectl wait --for=delete ns/<name>`, then apply again.
Requests fail during a rollout	Check that the `preStop` hook is present in the deployment (Lab 13).
`dvc` error about `_DIR_MARK`	Incompatible `pathspec`. Run `pip install pathspec==0.12.1` (already pinned).
`make drift-check` flags drift on normal traffic	Old drifted predictions are still in the log. Delete `logs/predictions.jsonl` first.

C. Glossary

Term	Meaning
CI (Continuous Integration)	Automatically lint, test and build every change.
CD (Continuous Delivery / Deployment)	Every verified change is ready to release (delivery) or released automatically (deployment).
CT (Continuous Training)	Automatically retraining and re-validating ML models.
Container image	A packaged, immutable copy of an app and everything it needs.
Registry	A store for images, e.g. GitHub Container Registry (GHCR).
Pod	The smallest unit Kubernetes runs: one or more containers.
Deployment / ReplicaSet	Keeps a desired number of identical pods running and manages rollouts.
Service	A stable address that load-balances across matching pods.
Liveness / readiness probe	Health checks: 'restart me if this fails' / 'send me traffic only when this passes'.
HPA	Horizontal Pod Autoscaler: adds or removes pods based on load.
PDB	PodDisruptionBudget: keeps a minimum number of pods up during maintenance.
ConfigMap	Configuration stored separately from the image.
Kustomize overlay	Environment-specific changes applied on top of a shared base.
IaC	Infrastructure as Code: defining infrastructure in version-controlled files.
Terraform state	Terraform's record of which real resources match your code.
GitOps	Git is the source of truth, and an in-cluster agent applies it.
Quality gate	An automatic check that blocks a release that doesn't meet a bar.
Experiment tracking	Recording the parameters, metrics and artifacts of each training run.
Model registry	Versioned storage of trained models with aliases like `champion`.
Data drift	Production inputs no longer look like the training data.
Concept drift	The relationship between inputs and the correct answer changes.
KS test	Kolmogorov–Smirnov test: measures how different two distributions are.
Canary release	Sending a small share of traffic to a new version first.
Rollback	Returning to the previous known-good version.
SLI / SLO / error budget	What you measure / the target for it / how much failure the target allows.
Postmortem	A blameless write-up of an incident and how to prevent it.

D. Progress tracker

Print this page or copy it into your notes. Tick each lab only when its checkpoint passes.

Done	Lab	Title	Date	Notes
☐	00	Setup & Git Workflow		
☐	01	Run, Test and Understand the App		
☐	02	Docker: Images, Layers, Multi-stage, Security		
☐	03	Docker Compose: A Multi-service Stack		
☐	04	CI/CD with GitHub Actions		
☐	05	Kubernetes Fundamentals		
☐	06	Kustomize Environments, Config and Scaling		
☐	07	Infrastructure as Code with Terraform		
☐	08	Experiment Tracking & Model Registry (MLflow)		
☐	09	Reproducible Pipelines & Data Versioning (DVC)		
☐	10	Continuous Training (CT)		
☐	11	Observability: Metrics, Dashboards, Alerts		
☐	12	Data Drift Detection & the Retraining Loop		
☐	13	Release Strategies: Rolling, Canary, Rollback		
☐	14	GitOps with Argo CD		
☐	15	Capstone: Incident Drills & Chaos Engineering		

Where to go next

- **Cloud:** rebuild the platform on EKS, GKE or AKS with Terraform (network, cluster, IAM, registry).
- **Serving at scale:** KServe, Seldon, BentoML or Ray Serve; GPU nodes; request batching.
- **Pipelines and features:** Airflow, Kubeflow or Prefect; Feast feature store.
- **LLMOps:** prompt versioning, evaluation harnesses, guardrails, token-cost monitoring.
- **Certifications:** CKA/CKAD, Terraform Associate, a cloud associate certification.